Lesson notes · for you

Data structures

Programming · Level 4 · Lesson 3

Time
20 min

Materials
A phone with this lesson downloaded; A pile of books or plates for a stack (optional)

Learning goals

- Choose a list, stack, queue, set or dictionary by the operations a job needs.
- Use a stack to check brackets.
- Count with a dictionary.

Activity: Activity: Bracket stack

1. Warm-up (3 min): Pile three books and take them off: last on, first off. Name a phone feature that works this way.
2. Check (7 min): Read “([{}])” one character at a time. Put each opening bracket on a pile of paper slips; at each closing one, take the top slip and check it matches.
3. Break (7 min): A naive checker only counts that there are as many (as). Write strings that fool it, such as “)(”, and check them with your stack.
4. Code (3 min): Run your strings through balanced() on the phone. Did the stack catch them all?

Paper slips make a perfect stack: only the top one can be taken.

Discussion

- Why is checking a set so much faster than checking a list?
- When would a queue be the wrong choice?
- What does a set give up in exchange for its speed?

Check yourself

- Name the structure for each: a browser’s back button, a printer’s jobs, a word count.

Try it this week
Count the letters in your full name with a dictionary: for each letter, count[c] = count.get(c, 0) + 1. Which letter appears most?

Worksheet

Data structures

Programming · Level 4 · Lesson 3

Name _____ Class _____ Date _____

1 **Your program checks, thousands of times, whether a student ID is registered. Which structure should hold the registered IDs?**

◯ A list ◯ A set ◯ A queue

2 **Write balanced(text) with a stack: it returns True if every bracket ([{ is closed by its partner in the right order, and False otherwise. Other characters are ignored.**

```
def balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    # push each opening bracket;
    # at each closing one, pop and check its partner
    return True
```

Write your program:

3 **Fill the gaps: the program counts the votes with a dictionary and prints Ali's count, 2.**

```
votes = ["Ali", "Bo", "Ali"]
count = {}
for v in votes:
    count[v] = count._____(v, _____) + 1
print(count["Ali"])
```

Word bank: `0` `get` `pop` `1`

4 **What does this program print?**

```
q = ["Amina", "Kofi", "Layla"]
q.append("Omar")
first = q.pop(0)
print(first)
```

◯ Amina ◯ Omar ◯ Layla

5 **Match each job to the best structure.**

Undo history	• • • •	Dictionary
Customers served in order of arrival	• • • •	Set
Is this ID registered?	• • • •	Queue
A pupil’s phone number, by name	• • • •	Stack

✎ **Think of a system at your school or university (the library, the canteen, the timetable). Which operations does it do most, and which structures would you choose?**